A VISUAL PRIMER · LLM SYSTEMS

Retrieval-Augmented *Generation*

RAG lets a language model answer using *your* documents instead of only what it memorized during training. It looks things up first, then writes — like an open-book exam rather than a closed-book one.

01 · WHAT IT IS & WHY WE USE IT

The model's memory is fixed. *Your knowledge isn't.*

A plain LLM only knows what was in its training data, frozen at a cutoff date. It can't see your company wiki, last week's tickets, or a PDF you uploaded an hour ago — and when it doesn't know, it tends to confidently invent an answer. RAG fixes this by fetching relevant text from a knowledge source and handing it to the model as context, so the answer is grounded in real, current, citable material.

PLAIN LLM

Closed book

- **Stale:** knows nothing past its training cutoff
- **No private data:** can't see your docs, DB, or internal wiki
- **Hallucinates:** fills gaps with plausible-sounding fiction
- **No sources:** can't show where an answer came from

RAG-POWERED LLM

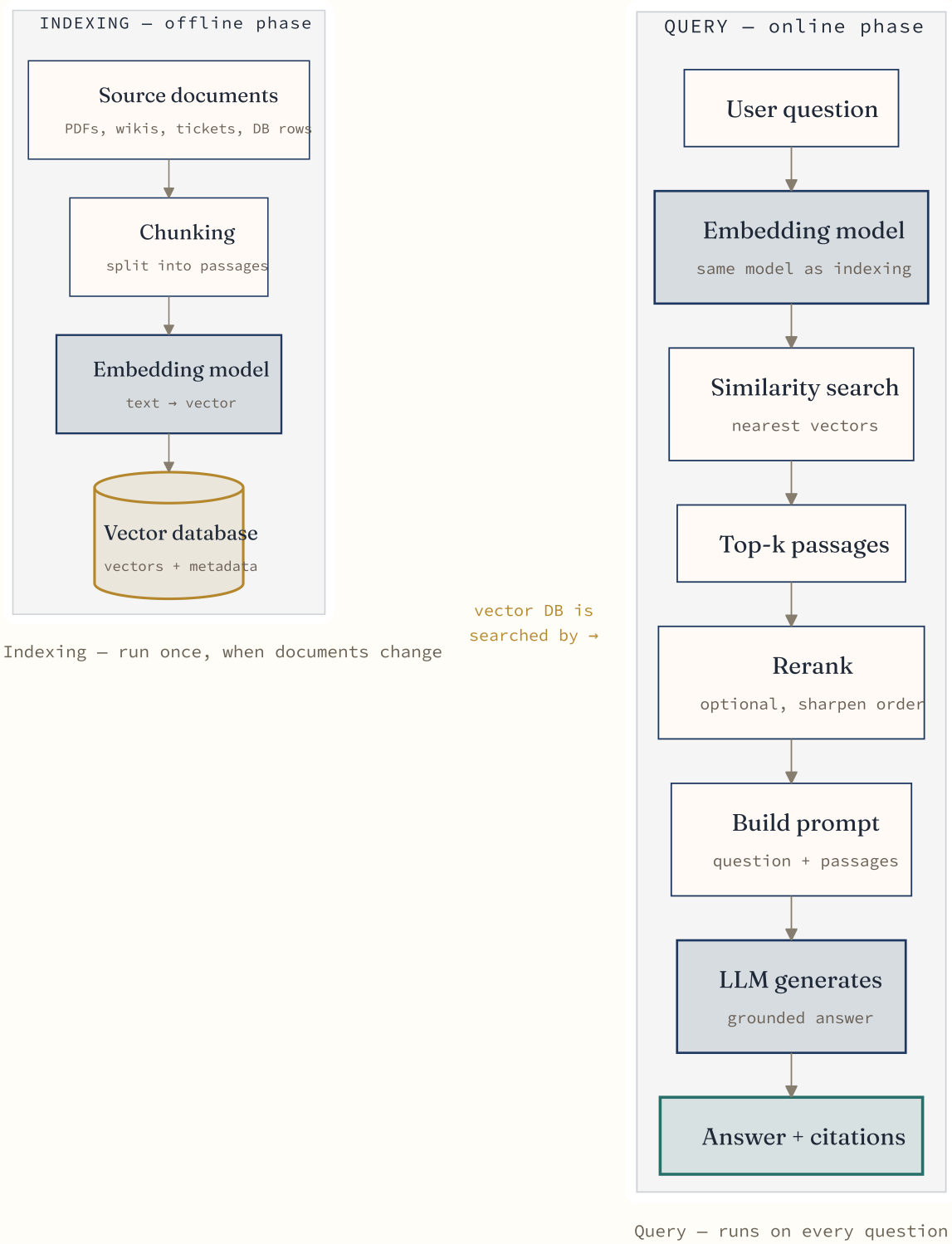
Open book

- **Fresh:** update the knowledge base, not the model
- **Grounded:** answers cite retrieved passages
- **Private-aware:** reads your own corpus on demand
- **Cheaper than fine-tuning:** no retraining to add facts

02 · END-TO-END WORKFLOW

Two phases: *index once,* retrieve every time

RAG splits into an offline **indexing** phase (done ahead of time, repeated only when documents change) and an online **query** phase (runs on every user question). The two meet at the vector database.



THE ONE RULE THAT TRIPS EVERYONE UP

The **same embedding model** must encode both your documents (indexing) and the user's question (query). Mixing models puts them in incompatible vector spaces, and similarity search returns garbage.

03 · WHAT HAPPENS BEHIND THE SCENES

One question, step by step

1

Embed the question

The user's question runs through the embedding model and becomes a vector — a list of numbers capturing its meaning.

2

Search the vector database

That query vector is compared against every stored document vector to find the closest matches by meaning, not keywords.

3

Retrieve top-k passages

The k nearest chunks (say, the top 5) are pulled out. These are the most relevant snippets from your corpus.

4

Rerank *(optional)*

A second, more precise model re-scores those candidates and reorders them, pushing the truly best passages to the top.

5

Assemble the prompt

The retrieved passages are stitched into the prompt alongside the question, with an instruction like "answer using only this context."

6

Generate the answer

The LLM reads question + context and writes a grounded reply — ideally citing which passages it used.

04 · KEY INTERNAL CONCEPTS

The vocabulary you'll actually *need*

CHUNKING

Splitting documents into passages

Long documents get cut into smaller pieces (a few hundred words each) so retrieval returns focused, relevant context instead of an entire 80-page manual. Overlapping chunks avoid cutting an idea in half at a boundary.

```
"Annual report.pdf" → 240 chunks of ~400 words, 50-word overlap
```

EMBEDDINGS

Turning text into vectors

An embedding model maps a chunk of text to a list of numbers (e.g. 1,536 dimensions). Text with similar *meaning* lands close together in this space — "car" and "automobile" end up as neighbors.

```
"refund policy" → [0.021, -0.44, 0.18, … ] (1536 dims)
```

VECTOR DATABASE

Where vectors live

A store built to hold millions of vectors and find nearest neighbors fast, using indexes like HNSW or IVF. It keeps each vector's original text and metadata (source, page, date) alongside it for filtering and citation.

```
stores: vector + text + {source, page, tenant_id}
```

SIMILARITY SEARCH

Finding the closest meaning

Comparing the query vector to stored vectors by distance — usually *cosine similarity*. Smaller distance means closer meaning. This is semantic, so it matches paraphrases that keyword search would miss.

```
cosine(query, chunk) → score between -1 and 1
```

TOP-K RETRIEVAL

Keep only the best handful

Rather than dumping everything, retrieval returns the *k* highest-scoring chunks (commonly 3–10). Small k keeps the prompt focused and cheap; too small risks missing the answer; too large adds noise.

```
k = 5 → 5 passages fed to the model
```

RERANKING

A second, sharper opinion

Vector search is fast but coarse. A cross-encoder reranker reads each candidate *together with* the question and re-scores them precisely — so you can retrieve top-20 cheaply, then rerank down to the best top-5.

```
retrieve 20 → rerank → keep 5 best
```

---

05 · BUILDING A RAG SYSTEM

The components you'll wire together

A working RAG pipeline is these moving parts in sequence. Get the first three right and most quality problems disappear.

COMPONENT	JOB	WATCH OUT FOR
Loader / parser	Pull text out of PDFs, HTML, docs, databases	Bad extraction (broken tables, OCR noise) poisons everything downstream
Chunker	Split text into retrievable passages	Chunk size + overlap is the biggest quality lever — tune it
Embedding model	Encode chunks & queries into vectors	Use one model everywhere; re-embed everything if you switch
Vector database	Store vectors, run fast nearest-neighbor search	Add metadata filters early (tenant, date, permissions)
Retriever	Turn a query into top-k passages	Hybrid (vector + keyword) beats pure vector for exact terms & IDs
Reranker (optional)	Reorder candidates for precision	Adds latency & cost — worth it when recall is high but ranking is messy
Prompt builder	Combine context + question + instructions	Tell the model to say "I don't know" when context is missing
LLM (generator)	Write the grounded answer	Mind the context window — too many chunks get ignored or truncated

06 · PRACTICAL TOOLS

What people actually *reach for*

VECTOR DATABASES

TOOL	STYLE	BEST WHEN…
pgvector Postgres extension	Self-hosted	You already run Postgres and want vectors next to your relational data — no new system to operate. Great for small-to-mid scale and per-tenant isolation.
Qdrant Purpose-built, open source	Self / Cloud	You want a dedicated vector engine with rich metadata filtering and good performance, self-hosted or as managed cloud.
Pinecone Fully managed SaaS	Managed	You'd rather not run infrastructure at all — scales to billions of vectors with low ops overhead, billed as a service.

EMBEDDING MODELS

MODEL / FAMILY	TYPE	NOTES
OpenAI embeddings text-embedding-3-small / -large	API	Strong quality, dead simple to call, pay per token. No GPU to manage, but data leaves your machine and you pay per request.
Open-source / local e.g. BGE, E5, GTE, Nomic, via sentence-transformers or Ollama	Self-hosted	Run on your own hardware — free at inference, fully private, no per-call cost. Needs some compute and setup; top open models now rival API quality.

RULE OF THUMB

Prototyping on a stack you already run? **pgvector + a local embedding model** gets you a private RAG system with zero new services. Need scale or zero ops? Reach for **Pinecone + an API embedding model**. Most teams land somewhere between, often on **Qdrant**.

> ***RAG doesn't make the model smarter — it makes the model better informed.***

That's the whole idea. You don't retrain a giant model every time a fact changes; you update a searchable knowledge base and let retrieval feed the model exactly what it needs, when it needs it. Everything else — chunking strategy, which vector DB, whether to rerank — is tuning around that core loop.

`Retrieval-Augmented Generation · a visual primer`

`Index once → retrieve per query · chunk → embed → store → search → rerank → generate`